Contents

1. **Single Layer Neural Networks**. We look at how a single layer network performs on a toy dataset
2. **Multilayer Perceptrons**. We look at how multiple layers perform on toy datasets
3. **Handwritten digit recognition in sklearn**. Learning to classify numbers using the libraries in sklearn
4. **Handwritten digit recognition in Pytorch**. Learning to classify numbers using libraries in Pytorch

Finally, try working through some tutorials in Pytorch and play around with the different hyperparameter choices. Links to tutorials are at the end of the notebook.

Single Layer Neural Network

In this question we apply a single-layer neural network to a linearly separable toy data set.

In [ ]:
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_moons, make_circles, make_classification
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import Perceptron
```

We start by generating and visualizing some data

In [ ]:
```
X1, y1 = make_classification(n_features=2, n_redundant=0, \
            n_informative=1, random_state=1,n_clusters_per_class=1)
fig1, ax1 = plt.subplots();
ax1.scatter(X1[:,0],X1[:,1],c=y1, cmap='rainbow')
plt.show()
```

Notice that the two classes can clearly be separated by a single line. We can now fit a single layer neural network to the data.

In [ ]:
```
nn1=Perceptron(alpha=1, max_iter=1000)
model=nn1.fit(X1,y1)
```

(Optional) Plot the line that the perceptron is modelling. Look at the documentation to find out how you can get the values of the weights and the bias.

In [ ]:
```
#TODO
```

We can visualise the performance of this network again using a scatter plot, and colour the points using predicted class. Is the network able to give the right classification for each point?

In [ ]:
```
ypred1=model.predict(X1)
# TODO: Make a scatter plot of the predictions.

# How do the predictions compare with the ground truth? Compute the accuracy t
```

Multi-Layer NN on Toy Problems

In this question we consider two toy problems in which the classes are not linearly separable. In the first example the two classes form moon shapes.

In [ ]:
```
X2, y2 = make_moons()
fig2, ax2 = plt.subplots()
ax2.scatter(X2[:,0],X2[:,1],c=y2, cmap='rainbow')
```

Try fitting a single-layer neural network to the data. Comment on the performance and hypothesise what might be the source of the errors.

In [ ]:
```
#TODO
```

We can now try to fit an multi-layer NN to the same data.

In [ ]:
```
nn2=MLPClassifier(alpha=1,hidden_layer_sizes=(10,10,10,10), max_iter=1000)
model2=nn2.fit(X2,y2)
ypred2=model2.predict(X2)
ax2.scatter(X2[:,0],X2[:,1],c=ypred2, cmap='rainbow')
fig2
```

This NN has 4 hidden layers each with 10 neurons. The parameter `hidden_layer_sizes=(x,y,z, ...)` identifies the number of neurons in each layer. Try experimenting with different configurations of hidden layers to see the effect on performance.

Investigate the parameter `activation`. What are the different activation functions?

The activation functions are:

We now introduce another toy classification problem based on classes in the form of two circles. Experiment with using different NN architectures to fit this data and plot the results.

In [ ]:
```
X3, y3 = make_circles()
fig3, ax3 = plt.subplots()
ax3.scatter(X3[:,0],X3[:,1],c=y3, cmap='rainbow')
plt.show()
```

In [ ]:
```
#TODO
```

Handwritten Digit Recognition

The data set contains images of hand-written digits. There are 10 classes where each class refers to a digit. Preprocessing programs were used to extract normalized bitmaps of handwritten digits from a preprinted form. 32 × 32 bitmaps are divided into non-overlapping blocks of 4 × 4 and the number of on pixels are counted in each block. This generates an input matrix of 8 × 8 where each element is an integer in the range 0...16.

In [ ]:
```
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.datasets import load_digits
digits = load_digits()
```

Divide the data into training and test sets, fit a NN to the training data and evaulate its accuracy on both training and test data.

In [ ]:
```
#TODO
```

It is also useful to plot a time series graph of the loss function (error) against iteration.

In [ ]:
```
loss_values = model4.loss_curve_
plt.figure()
plt.plot(loss_values)
plt.show()
```

Try experimenting with different architectures and activation functions.

Handwritten digit recognition in Pytorch

This is an excerpt from a tutorial in Pytorch, available at https://docs.pytorch.org/tutorials/index.html. Pytorch has lots of tutorials if you want to investigate.

In this tutorial, you are asked to:

1. Load a prebuilt dataset.

2. Build a neural network machine learning model that classifies images.
3. Train this neural network.
4. Evaluate the accuracy of the model.

We firstly import Pytorch. You may need to install Pytorch first.

In [ ]:
```
import torch
print("PyTorch version:", torch.__version__)
```

Load the MNIST digits dataset

Load the dataset and convert data from integgers into floating point numbers:

In [ ]:
```
from torchvision.datasets import MNIST
from torchvision.transforms import ToTensor
import numpy as np

# Load MNIST
train_ds = MNIST(root="./data", train=True, download=True, transform=ToTensor(
test_ds  = MNIST(root="./data", train=False, download=True, transform=ToTensor

# Convert to numpy (to match your existing code style)
x_train = train_ds.data.numpy() / 255.0
y_train = train_ds.targets.numpy()

x_test = test_ds.data.numpy() / 255.0
y_test = test_ds.targets.numpy()
```

Build a machine learning model

Here, the model is a sequential model, meaning that it has a feed-forward architecture rather than a recurrent architecture or otherwise.

We set up a Sequential model and then add layers to it. Each layer can have different types: Flatten, Dense, Dropout. Look through the documentation for each layer to find out about them.

In [ ]:
```
import torch.nn as nn

model = nn.Sequential(
    nn.Flatten(),          # (28, 28) -> 784
    nn.Linear(28 * 28, 256),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(256, 10)
)
```

For each datapoint, the model returns a vector of logits or log-odds scores, one for each class. Below, we make predictions for just one datapoint.

In [ ]:
```
model.eval()  # set eval mode (important for Dropout)

with torch.no_grad():
    x = torch.tensor(x_train[:1], dtype=torch.float32)
    predictions = model(x).numpy()

predictions
```

The tf.nn.softmax function converts these logits to probabilities for each class - now we have the probabilities that this datapoint belongs to each class.

In [ ]:
```
torch.softmax(torch.tensor(predictions), dim=1).numpy()
```

For training the model, we need to define a loss function. We will use losses.SparseCategoricalCrossentropy, which takes a vector of logits and a True index and returns a scalar loss for each example.

In [ ]:
```
loss_fn = nn.CrossEntropyLoss()
```

This loss is equal to the negative log probability of the true class: The loss is zero if the model is sure of the correct class.

This untrained model gives probabilities close to random (1/10 for each class), so the initial loss should be close to `-tf.math.log(1/10) ~= 2.3`.

In [ ]:
```
y = torch.tensor(y_train[:1], dtype=torch.long)
pred = torch.tensor(predictions, dtype=torch.float32)

loss_fn(pred, y).item()
```

Before we start training, we compile the model, setting the kind of optimizer we want to use (`adam` , the loss function, as set up above, and the metric we want to measure (`accuracy`).

In [ ]:
```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

def accuracy(logits, y):
    preds = torch.argmax(logits, dim=1)
    return (preds == y).float().mean().item()
```

Train and evaluate your model

In [ ]:
```
model.train()
```

```
x = torch.tensor(x_train, dtype=torch.float32)
y = torch.tensor(y_train, dtype=torch.long)

epochs = 500
for epoch in range(epochs):
    logits = model(x)
    loss = loss_fn(logits, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    acc = (torch.argmax(logits, dim=1) == y).float().mean().item()
    print(f"Epoch {epoch+1}/{epochs} - loss: {loss.item():.4f} - acc: {acc:.4f
```

Checks the model's performance

In [ ]:
```
model.eval()

x = torch.tensor(x_test, dtype=torch.float32)
y = torch.tensor(y_test, dtype=torch.long)

with torch.no_grad():
    logits = model(x)
    loss = loss_fn(logits, y)
    acc = (torch.argmax(logits, dim=1) == y).float().mean().item()

print(f"Test loss: {loss.item():.4f}")
print(f"Test accuracy: {acc:.4f}")
```

The model is now trained to almost 98% accuracy. We evaluated the mode on the test set. Can you evaluate it on the training set? Try altering the structure of the network that we set up. Can you change the performance? It will be quite difficult to improve on 98% - but see if you can make it worse! What sort of changes can you make? What effect do they have?

In [ ]:
```

```

In [ ]:
```

```